

基于两阶段复合扰动的模拟退火 VLSI 宏单元 与 I/O 引脚协同布图算法

陈安康
计算机学院, 中山大学
广州, 中国
1743826979@qq.com

摘要—随着超大规模集成电路 (VLSI) 设计进入后摩尔时代, 物理设计面临的挑战愈发严峻, 其中, 布图规划 (Floorplanning) 作为决定芯片最终性能、功耗和面积 (PPA) 的关键前端环节, 其重要性不言而喻。一个尤其突出且具有挑战性的问题是如何在布图早期阶段, 协同优化内部宏单元 (Macro) 的放置与外部 I/O 引脚 (Pin) 的排布, 以减少全局绕线压力和时序收敛难度。针对此问题, 本文提出了一种基于模拟退火 (Simulated Annealing, SA) 的协同优化布图算法框架。该框架的核心是一种创新的“两阶段复合扰动”策略: 在 SA 的每次迭代中, 首先通过对宏单元的序列对 (Sequence Pair) 表示进行随机扰动以探索新的布图拓扑; 随即, 一个确定性的 I/O 引脚重规划模块会立刻响应新拓扑, 动态地重新计算所有引脚的最佳位置。这种紧耦合的迭代方式保证了每一个解都是一个完整的、经过协同考量的布图方案。本文详细阐述了该算法的整体框架、两阶段动作的具体实现以及质量评估模型。最后, 我们在 GSRC, MCNC 基准电路上对该框架进行了全面的实验验证。

Index Terms—VLSI, 布图规划, 协同优化, 模拟退火, 序列对, I/O 引脚布图, 成本函数

I. 引言

A. 背景: 后摩尔时代的物理设计挑战

随着摩尔定律的物理极限日益临近, 单纯依靠缩小晶体管尺寸来提升性能的道路变得越发艰难。为了延续技术发展的步伐, 芯片设计正朝着异构集成、先进封装 (如 2.5D/3D-IC [1]) 和芯粒 (Chiplet) 等新范式演进。这些新技术使得单个封装内的系统变得空前复杂, 成百上千的宏单元 (如处理器核、内存、专用加速器) 和 I/O 接口需要被高效地集成。在这样的背景下, 物理设计, 特别是作为其开山之作的布图规划 (Floorplanning), 其重要性被提升到了前所未有的战略高度。一个优秀的布图方案不仅是实现芯片 PPA (Performance, Power,

Area) 目标的基石, 更是决定整个设计项目成败的关键。

B. 研究动机: 协同优化的缺失与挑战

布图规划的核心任务是在芯片上为所有宏单元找到一个最佳的、无重叠的物理位置, 以优化一系列相互冲突的目标, 如最小化面积和总线长。该问题已被证明是 NP 难问题, 因此学术界和工业界长期致力于开发高效的启发式算法, 如模拟退火 (SA) [2]、遗传算法 [3]、蚁群算法 [4] 等。

尽管现有的布图算法已取得巨大成功, 但我们观察到在处理一个关键问题上仍存在普遍的简化和不足: 宏单元布图与 I/O 引脚布图的协同优化。

观察到的问题: 传统的布图流程往往专注于宏模块部署, 将 I/O 引脚的分配视为一个独立的、后置的处理步骤。即先完成核心宏单元的布图, 形成一个固定的“大陆板块”, 然后在布局阶段 (Placement) 时将 I/O 引脚“按”到这个板块的边缘。这种解耦的设计方法不利于对布图进行质量评估, 因为其在宏模块布图时并没有考虑到 I/O 引脚的布图需求。尤其是 I/O 引脚对宏模块布图的影响。

近年来, 虽然有基于深度学习 [5]、强化学习 (DRL) [6] 或解析法 [7] 的先进布图方法, 但它们或需要大量训练数据, 或在处理硬宏单元和动态 I/O 约束时仍面临挑战。

我们的思考与改进动机: 基于上述情况, 我们提出: 能否在布图阶段就动态考虑到 I/O 的布置, 设计一个能将两者紧密耦合的、高效的启发式优化框架? 这便是本文研究的核心动机。

C. 我们的方法与贡献

为应对上述挑战,本文针对一种特定的 Floorplan 协同布图优化问题,引入一种创新的“两阶段复合扰动”策略来解决协同优化问题。在搜索算法的每一次迭代中,首先通过对宏单元的序列对 (Sequence Pair) 表示进行扰动,探索新的宏单元布图拓扑。紧接着,一个确定性的 I/O 引脚重规划模块会立刻响应这个新拓扑,根据宏单元连接的“重心”和边缘可用性,动态地重新计算出所有 I/O 引脚的最佳位置。

本文的主要贡献可以总结如下:

- 提出了一种将宏单元布图与 I/O 引脚布图紧密耦合在单次迭代中的协同优化思想,以应对传统分离式方法的局限性。
- 设计并实现了一个完整的、基于模拟退火和序列对表示的两阶段协同布图器。留下一个可扩展的框架,便于未来集成更复杂的评估模型和优化策略。
- 在标准的 GSRC, MCNC [8] 基准电路上,对该框架的有效性进行了全面的实验验证和分析,证明了其在生成高质量布图方案上的能力。

D. 论文结构

本文的其余部分组织如下:第二节对本文讨论的协同布图问题进行形式化描述。第三节详细阐述了我们提出的两阶段模拟退火协同优化算法框架。第四节介绍了布图质量的评估模型。第五节展示并分析了在 GSRC, MCNC 基准电路上的实验结果。最后,第六节对本文工作进行了总结并展望了未来的研究方向。

II. 问题描述

VLSI 宏单元与 I/O 引脚协同布图问题可以形式化地描述如下:

• 输入:

- 1) 一组宏单元集合 $M = \{m_1, m_2, \dots, m_n\}$ 。每个模块 m_i 具有固定的宽度 w_i 和高度 h_i 。
- 2) 一个需要被放置的 I/O 引脚集合 $P = \{p_1, p_2, \dots, p_l\}$ 。
- 3) 一个网络集合 $N = \{n_1, n_2, \dots, n_k\}$, 定义了模块与模块之间、模块与 I/O 引脚之间的连接关系。

• 输出:

- 1) 每个模块 m_i 的坐标 (x_i, y_i) 。

- 2) 每个 I/O 引脚 p_j 的坐标 (x_j, y_j) 。

• 约束与目标:

- 1) **非重叠约束:** 任意两个模块 m_i 和 m_j 在物理上不能重叠。
- 2) **边界约束:** 所有 I/O 引脚 p_j 必须位于由所有宏单元布图决定的芯片包围盒的边缘上。
- 3) **网格化:** 所有模块与引脚坐标均为整数。
- 4) **优化目标:** 最小化一个综合成本函数 $Cost$, 综合考虑芯片面积 (Area)、Feedthrough 成本 (穿越成本)、总线长 (Wirelength)、合法性成本的加权和。

$$\phi = \alpha \frac{A_{chip}}{A} + \beta \frac{WL_{hpwl}}{WL} + \gamma_n \frac{F_{num}}{F_{num}} + \gamma_w \frac{F_{wl}}{F_{wl}} + \delta \frac{R_{ws}}{R_{ws}} + \zeta \frac{P_{illegal}}{P_{illegal}} \quad (1)$$

其中 α, β 是用户定义的权重系数。

III. 协同优化的布图算法框架

我们设计了一个以模拟退火 (SA) 为外部驱动的协同优化框架。该框架通过一个复合扰动动作,在单次迭代中同时更新宏单元和 I/O 引脚的布图,从而实现紧耦合优化。本章将详细阐述该框架的三个关键组成部分:布图表示法、两阶段复合扰动,以及作为优化引擎的模拟退火流程。

A. 布图表示

布图 (Placement) 是超大规模集成电路 (VLSI) 物理设计中最为核心的步骤之一。其根本目标是在给定的芯片区域内,为电路中的所有宏单元 (Macro) 和标准单元 (Standard Cell) 确定最佳的二维坐标,从而在满足物理约束的前提下,最小化总线长 (Wirelength)、关键路径延迟 (Timing Delay) 和功耗,并最终优化芯片的性能、功耗与面积 (PPA)。由于宏单元数量巨大且布局方案的可能性呈指数级增长,布图问题是一个典型的 NP 难问题,其解空间极其庞大。为了在如此复杂的空间中高效地寻找高质量的解,学术界与工业界发展出了多种经典的算法思想。这些方法主要可分为两大类:一类是基于迭代优化的启发式算法,如模拟退火 (Simulated Annealing) 和基于分割 (Partitioning-Based) 的算法,它们通过不断地对布局进行微调并评估其成本来逐步改进方案;另一类是解析法 (Analytical Placement),

它将布图问题转化为连续的数学优化问题进行求解。然而，无论是何种优化引擎，其有效运行的基础都依赖于一个能高效描述和操作单元之间拓扑关系的编码方案。

在众多的布图表示法中，非切片（Non-slicing）表示法因其能够覆盖更广泛、更优质的解空间而成为现代布图算法的主流选择。相较于早期的切片（Slicing）结构因其拓扑限制而无法表达所有可能的紧凑布局，非切片表示法具备更强的灵活性和完备性。因此，我们采用其中一种经典的非切片表示法——序列对（Sequence Pair, SP）[9]。该方法能表示任意的矩形打包布图，解空间完备，不会因表示法的局限而错失最优解。该方法能表示任意的矩形打包布图，解空间完备，不会因表示法的局限而错失最优解。一个包含 n 个宏单元的布图方案被唯一地编码为一对模块索引的排列 (Γ_+, Γ_-) 。这两个排列通过以下几何约束，完全确定了所有模块间的相对位置拓扑：

- 若 $a \prec_+ b$ 且 $a \prec_- b$ ，则 a 在 b 的左侧。
- 若 $a \prec_+ b$ 且 $b \prec_- a$ ，则 a 在 b 的下方。

为了从序列对解码出具体的坐标，需要构建两个加权有向无环图（DAG）：一个水平约束图 G_H 和一个垂直约束图 G_V 。图中每个节点代表一个宏单元，权重为其相应的宽度或高度。通过计算这两个 DAG 中的最长路径，即可在 $O(n^2)$ 时间复杂度内确定所有模块的坐标，并得到一个最紧凑的布图。

B. 核心思想：两阶段复合扰动

在 SA 的每次迭代中，我们通过一个复合扰动动作（GenerateNewSolution）来生成新的完整布图方案，而非仅改变宏单元的位置。这一复合动作是实现协同优化的关键，它包含两个紧密衔接的阶段，确保了 I/O 引脚的布图能即时响应宏单元布图的变化。

1) 阶段一：宏单元布图拓扑扰动：此阶段是算法探索新解空间的主要动力，它直接作用于序列对表示上。典型的扰动操作包括：

- 1) **交换元素**：在 Γ_+ 或 Γ_- 中，随机选择并交换两个模块的位置。
- 2) **模块旋转**：随机选择一个宏单元，将其旋转 90 度。

2) 阶段二：基于重心的 I/O 引脚确定性重规划：此模块是协同优化的精髓所在。它是一个响应宏单元布图变化的、高效的确定性过程。该模块规定了三个阶段，比较阶段、扩增阶段、排布阶段。考虑到问题中坐标非

连续的约束，在计算得到宏单元组成的布图包围盒后，需要判断引脚是否可以全部放在包围盒的边缘上。若不能，则需要进入扩增阶段进行芯片的扩增，直到所有的 I/O 引脚都可以放在芯片边缘。此时进入排布阶段。

排布阶段中可以随意实现其他的算法。我们选择了一种高效的、基于**重心法（Center of Gravity）**的贪心算法来快速地为 I/O 引脚找到一个局部最优的排布。其贪心策略是优先放置连接网络数最多的引脚。对于每个引脚 P_i ，我们计算其连接网络中所有宏单元的重心 G_i ，其公式为：

$$G_i = \frac{1}{|\mathcal{M}_i|} \sum_{m_k \in \mathcal{M}_i} C(m_k) \quad (2)$$

其中 $\mathcal{M}_i$ 是与 P_i 相连的所有宏单元的集合， $C(m_k)$ 是宏单元 m_k 的中心坐标。算法随后在芯片边缘所有可用的整数坐标“插槽”中，为 P_i 选择离其重心 G_i 欧氏距离最近的可用位置。

C. 两阶段复合扰动模拟退火流程

我们的优化引擎基于一个**协同优化（Co-optimization）**的模拟退火（SA）框架，旨在同时完成宏单元（Macro）布局和 I/O 引脚规划。该方法的核心思想是在同一个退火流程中，将两者紧密耦合，以实现全局最优。与将宏单元和 I/O 引脚分步优化的传统方法不同，我们的算法在每次迭代中都动态地调整两者，确保 I/O 引脚的布局能够实时适应宏单元位置的变化。

算法的整体流程如算法 1 所示。在初始化阶段，我们首先生成一个初始的宏单元布局，并随即调用 I/O 引脚规划模块为其分配初始引脚位置，该初始方案被同时记录为当前解 $S_{current}$ 和最优解 S_{best} 。

随后，算法进入主退火循环。该循环采用经典的**几何降温策略**（ $T \leftarrow T \times \text{cooling_rate}$ ），温度从一个较高的初始值 $T_{initial}$ 单调递减，直至低于预设的最终温度 T_{final} 。在每次迭代中，我们首先对当前的宏单元布局施加一个扰动（‘MacroPerturbation’）产生一个新的临时布局。一个关键的步骤是，我们**立即**基于这个新的宏单元布局，重新运行 I/O 引脚规划模块（‘PinPlanModule’），从而生成一个与之匹配的、新的 I/O 引脚方案。这种协同优化的方式保证了每次成本评估都是针对一个完整的、宏单元与 I/O 引脚相协调的布局方案。

在评估新方案之前，我们会检查是否存在宏单元重叠，任何导致重叠的无效移动都会被直接舍弃。对于有效移动，我们计算其成本函数，并遵循 **Metropolis 准则**：如果新方案成本更低 ($\Delta Cost < 0$)，则无条件接受；否则，以 $e^{-\Delta Cost/T}$ 的概率接受这一非更优解。这种概率性接受机制是 SA 算法能够有效跳出局部最优、进行全局探索的关键。在整个退火过程中，我们会持续追踪并更新所遇到的历史最优解 S_{best} ，并在退火结束后将其作为最终结果返回。

Algorithm 1 基于模拟退火的协同优化楼层规划器

Input: 宏模块尺寸信息 (M), 线网连接关系 (N), I/O 引脚总数 ($NumPins$)

Output: 优化后的最佳布图方案 S_{best} (包含宏模块和 I/O 引脚的坐标)

```

1: Load  $M$ ,  $N$ , and  $NumPins$  from input files (.blocks, .nets)
2:  $S_{macros\_best} \leftarrow \text{INITIALMACROPLACEMENT}(M)$ 
3:  $S_{pins\_best} \leftarrow \text{PINPLANMODULE}(S_{macros\_best}, NumPins)$ 
4:  $S_{best} \leftarrow (S_{macros\_best}, S_{pins\_best})$ 
5:  $S_{current} \leftarrow S_{best}$ 
6:  $Cost_{current} \leftarrow \text{EVALUATECOST}(S_{current})$ 
7:  $Cost_{best} \leftarrow Cost_{current}$ 
8:  $T \leftarrow T_{initial}$ 
9:  $T_{final} \leftarrow \text{final\_temperature}$ 
10:  $cooling\_rate \leftarrow \text{cooling\_rate\_value}$ 
11: while  $T > T_{final}$  do
12:    $S_{macros\_new\_temp} \leftarrow \text{MACROPERTURBATION}(S_{macros\_current})$ 
13:    $S_{pins\_new} \leftarrow \text{PINPLANMODULE}(S_{macros\_new\_temp}, NumPins)$ 
14:    $S_{new} \leftarrow (S_{macros\_new\_temp}, S_{pins\_new})$ 
15:   if  $\text{HASOVERLAP}(S_{macros\_new\_temp})$  then
16:     continue
17:   end if
18:    $Cost_{new} \leftarrow \text{EVALUATECOST}(S_{new})$ 
19:    $\Delta Cost \leftarrow Cost_{new} - Cost_{current}$ 
20:   if  $\Delta Cost < 0$  or  $\text{RANDOM}(0, 1) < e^{-\Delta Cost/T}$  then
21:      $S_{current} \leftarrow S_{new}$ 
22:      $Cost_{current} \leftarrow Cost_{new}$ 
23:     if  $Cost_{current} < Cost_{best}$  then
24:        $S_{best} \leftarrow S_{current}$ 
25:        $Cost_{best} \leftarrow Cost_{current}$ 
26:     end if
27:   end if
28:    $T \leftarrow T \times cooling\_rate$ 
29: end while
30: return  $S_{best}$ 

```

IV. 布图质量评估模型

布图质量的优化是衡量一个布图算法优劣的核心，也是整个物理设计流程成败的关键。为了有效地指导模拟退火 (Simulated Annealing) 算法在庞大的解空间中进行启发式搜索，我们必须能够精确地量化一个布图方案的“好”与“坏”。这通过一个精心设计的成本函数 (Cost Function) 来实现，它将一个布局方案的多个质量维度映射为一个单一的标量值，即该状态的“能量”。

模拟退火算法的本质，正是在于不断尝试降低这个能量值。

在现代 VLSI 设计中，布图是一个典型的多目标优化问题 (Multi-Objective Optimization)，单纯优化任何一个指标都可能导致其他指标的恶化。因此，我们的成本函数是一个综合了多项关键指标的加权和，旨在同时优化芯片的尺寸、互连效率和布线复杂性，在相互冲突的目标之间取得最佳平衡。

我们的评估器 (ComprehensiveEvaluator) 支持多项关键评估指标，包括面积 (Area)、总线长 (Wirelength)、Feedthrough 数量 (Feedthrough Number)、Feedthrough 线长 (Feedthrough Wirelength)、空白区域 (Whitespace) 和合法性 (Legality)。

—

A. 各项指标的简要说明

1) 面积 (Area): 布图的总面积是衡量芯片尺寸和制造成本的直接指标。它由所有宏单元的外包围盒决定。

$$\text{Area} = W_{\text{chip}} \times H_{\text{chip}}$$

其中， W_{chip} 和 H_{chip} 分别是包含所有模块的最小矩形区域的宽度和高度。更小的面积意味着更高的集成度和更低的制造成本，是我们追求的核心目标之一。

2) 总线长 (Wirelength): 总线长是评估互连复杂性、信号延迟和功耗的关键指标。我们主要采用半周长线长模型 (Half-Perimeter Wirelength, HPWL) [10] 进行估算。对于一个连接多个引脚的网络 n_j ，其 HPWL 计算方式为该网络所有引脚（包括模块内部引脚和 I/O 引脚）坐标所构成包围盒的半周长。

$$\text{HPWL}(n_j) = (\max_{i \in n_j} x_i - \min_{i \in n_j} x_i) + (\max_{i \in n_j} y_i - \min_{i \in n_j} y_i)$$

总线长成本 W 是所有网络 HPWL 的加权和（考虑网络权重）：

$$\text{Wirelength} = \sum_{j=1}^k \text{HPWL}(n_j)$$

最小化总线长有助于降低布线拥塞、减小信号传播延迟和降低功耗。此外，我们的线长计算模块 (WirelengthCalculator) 也支持矩形最小生成树 (RMST) [11] 和 Steiner 树 (Steiner Tree) [12] 的近似估算，提供更全面的线长评估能力。

3) Feedthrough 数量与线长 (Feedthrough Number & Wirelength): **Feedthrough (穿透)** 是指一条网络线为了连接其引脚，不得不穿过其他不属于该网络的模块的现象。这会显著增加布线难度，引入额外的过孔 (via) 和层转换，进而增加芯片面积、功耗和信号延迟。因此，最小化 Feedthrough 是布图优化的重要目标。

我们的评估模型将 Feedthrough 分为两个独立的指标：**Feedthrough 数量 (FTNUM)** 和 **Feedthrough 线长 (FTWL)**，并为其分配独立的权重，以更精细地控制优化方向。

4) 空白区域 (Whitespace): 空白区域是指芯片总面积中未被模块占用的部分。虽然一定量的空白是必要的（例如用于布线通道、缓冲区等），但过多的空白会浪费芯片面积，增加成本。我们关注空白区域比例 (whitespace_ratio) 及其碎片化程度 (fragmentation_index) 和可利用性，目标是合理分配空白，使其既能满足布线需求，又能保持高利用率。

5) 合法性 (Legality): 合法性是布图方案必须满足的硬性约束。任何不合法（例如模块重叠、超出芯片边界）的方案都是不可接受的，即便它们的面积或线长看起来很好。任何违规都会产生高额的惩罚代价，并根据违规的类型和严重程度进行加权，从而在优化过程中强烈地“推动”算法避开不合法的解。

—

B. 归一化校准 (Normalization Calibration)

不同评估指标的原始数值范围可能差异巨大（例如，面积可能是数十万，而空白比例是 0-1 之间）。如果直接相加，数值大的指标会主导总成本函数，导致优化器无法平衡优化所有目标 [13]。为了解决这个问题，我们引入了动态归一化校准机制。

工作原理: 在模拟退火优化开始之前，评估器会进入一个校准阶段 (calibrate 方法)。在此阶段，它会：

- 1) 生成一定数量 (calibration_samples) 的随机布图样本。
- 2) 对每个随机样本，计算所有评估指标的原始值。
- 3) 对每种指标，计算其在这些随机样本中的平均值。
- 4) 将这些平均值作为该指标的归一化因子。例如，`normalization_factors['area'] = np.mean(area_samples)`。

应用: 在实际评估 (evaluate) 过程中, 每个指标的原始值会被其对应的归一化因子除, 从而将所有指标都转换到一个相似的数值范围 (例如, 归一化后的值通常在 0.5 到 2.0 之间波动)。

$$\text{Normalized_Metric} = \frac{\text{Raw_Metric}}{\text{Normalization_Factor}}$$

然后, 这些归一化后的指标再乘以其对应的权重, 相加得到最终的总成本。这种动态归一化方法确保了不同指标的权重设置能够有效且公平地指导优化方向, 而不是被原始数值大小所影响。

C. Feedthrough 估算模型

我们提供了两种 Feedthrough 估算方法, 可根据需求在评估器中进行配置:

1) 原始简化方法 (*Original Simplified Method*): 这是在 `evaluation/metrics.py` 中 `calculate_feedthrough` 函数实现的简化方法。

- 原理: 对于每一个网络, 首先构建其所有引脚的曼哈顿距离最小生成树 (RMST) 的边。然后, 对于 MST 中的每一条边, 检查它在芯片上连接两个引脚的直线路径是否穿越了任何其他不属于该网络的模块。
- 简化处理: 如果一条线段穿越了一个模块, 则简单地将 Feedthrough 计数器加一。这种方法主要关注 Feedthrough 的数量 (FTNUM), 提供了一个快速但相对粗略的估算。

2) FTAFP 论文模型: 我们的 Feedthrough 估算模型基于 FTAFP 论文 (Section 3.1) [17] 提出的方法, 并出于性能考虑, 对 FTNUM 的评估方法进行简化。

FTAFP 模型主要由两部分组成:

网络简化 (Net Simplification) 这一阶段将复杂的网络分解为更小的子网络 (SubNet)。它通过并查集 (Union-Find) 算法将物理上邻接的模块归为一个子网络。每个 SubNet 会记录其包含的模块、中心坐标和包围盒信息。

Feedthrough 估算 (Feedthrough Estimation) 此阶段在简化的子网络图上进行 Feedthrough 估算。首先, 在子网络之间构建最小生成树 (MST)。MST 的边表示子网络之间主要的连接路径。对于 MST 的每

条边, 计算其对应的 Feedthrough 线长 (FTWL) 和 Feedthrough 数量 (FTNUM)。

$$\text{FTWL}(u, v) = \text{WL}_{\text{Manh}}(u, v) - \text{WL}_O(u) - \text{WL}_O(v)$$

其中 WL_{Manh} 是子网络 u 和 v 中心之间的曼哈顿距离, 而 $\text{WL}_O(s)$ 是子网络 s 内部的“优化线长”, 它基于子网络的包围盒 HPWL 和模块填充率来估计。这旨在从总线长中减去那些不需要穿透的内部布线。

FTNUM 估算: 对于连接两个子网络 u 和 v 的路径, 算法会检查这条路径的包围盒与所有不属于当前网络的宏模块 (障碍物) 的交集。

简化处理 (FTAFP 数量估算): FTAFP 方法采用了一种更复杂的函数来估算 FTNUM。它不只是简单地计算穿越的模块数量, 而是通过计算路径包围盒与障碍物的重叠总面积 (`overlap_area`) 及其重叠模块的数量 (`overlap_num`) 来推导。具体公式如下: 设 $r = \frac{\text{overlap_area}}{\text{path_bbox_area}}$, $\text{threshold} = 0.4$, $\beta = \frac{\text{overlap_num}}{1 - \text{threshold}}$ 。若 $r > \text{threshold}$, 则 $\text{Feedthrough_Count} = \beta \times (r - \text{threshold})$ 。这个公式表示, 只有当路径包围盒与障碍物的重叠比例 r 超过一定阈值 (`threshold`) 时, 才开始计算 Feedthrough 数量, 并且数量的增长与超过阈值的程度和重叠的模块数 `overlap_num` 成正比。这种非线性关系更好地模拟了真实世界中复杂布线环境下的穿透效应。

V. 实验结果与分析

A. 实验设置

为了评估我们实现的协同优化布图器, 我们使用了 GRSC 国际公认的 MCNC 基准电路集。整个算法框架使用 Python 实现。所有实验在一台配置为 Intel Core i9-12900K CPU @ 4.8GHz, 32GB RAM 的服务器上运行。

算法通过 YAML 配置文件进行参数调整, 并通过 ‘python main.py <config.yaml>’ 命令启动。

优化过程结束后, 程序会自动生成一个结果文件夹, 其中包含优化过程中的可视化布图图、一个记录最终布图各项质量评估指标的 JSON 文件, 以及一个符合标准格式的 ‘.pl’ 文件。

B. 实验场景与假设

为验证核心算法的有效性, 本次实验在以下场景假设下进行:

表 I

COMPARISON OF BASELINES WITH FTAFP, WITHOUT FEEDTHROUGH OPTIMIZATION. (ADDED COLUMN)

Benchmarks			Corblivar [14]	SP-FOFP [15]	CB-Tree [16]	FTAFP [17]	Model1
Case	# Modules	# Nets	HPWL	HPWL	HPWL	HPWL	HPWL
n10	10	118	47,899	43,071	42,007	40,778	32,339
n30	30	349	175,684	160,774	126,952	111,877	121,149
n50	50	485	208,356	193,478	165,783	162,011	184,986
n100	100	885	328,202	304,279	310,582	293,497	350,740
n200	200	1585	607,503	554,992	546,521	524,989	734,349
ami33	33	123	99,355	91,037	96,166	90,503	48,669
ami49	49	408	1,202,310	1,010,759	1,042,454	1,007,618	210,168

Layout Visualization - Final Solution

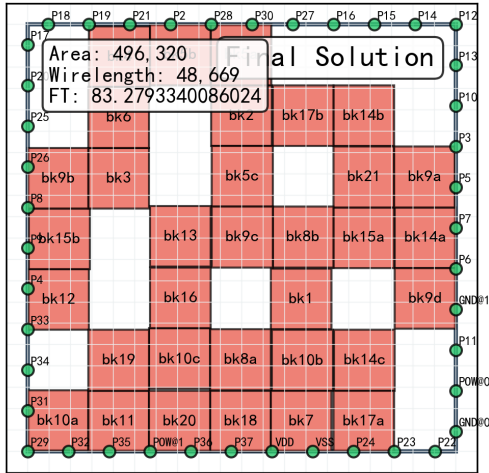


图 1. 在 ami33 电路上布图的结果示意图

Layout Visualization - Iteration 4000

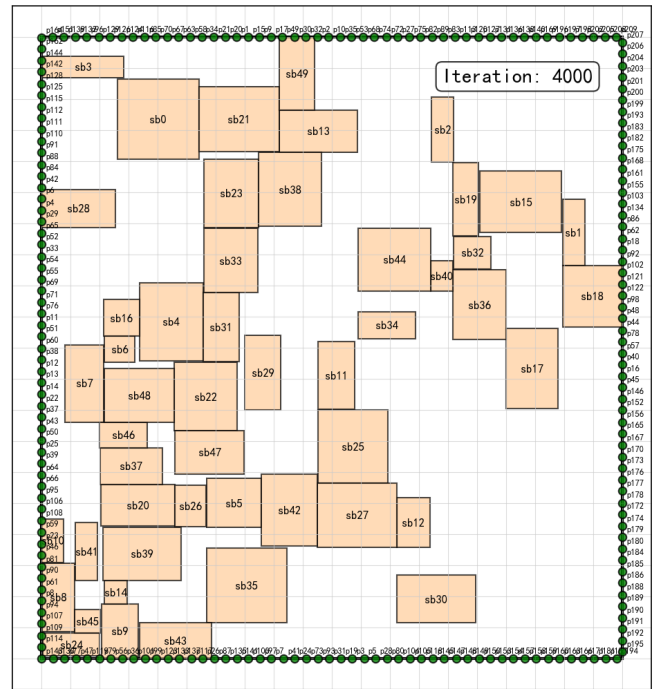


图 2. 在 n30 电路上布图的结果示意图

- **模块类型:** 所有模块均为矩形的硬宏单元，其尺寸在布图过程中保持不变。
- **坐标系:** 布图在整数网格上进行，所有模块和引脚的坐标都必须是整数。
- **芯片边界:** 芯片边界是灵活的，由所有宏单元的布图包围盒动态决定。
- **引脚模型:** 用于线长计算的模块引脚被简化并假设位于其所属模块的中心；I/O 引脚必须均匀精确地放置在芯片包围盒的最外层边缘上。

记本次实验模型为 Model1，首先与其他模型进行比较，结果参考 [17]。从表 I 可以看出，我们的算法能够为不同规模的电路生成优质的布图方案。在 HPWL 的比较中，我们的结果在中小规模电路表现出色，但在大规模电路上仍有提升空间。但可能是 SA 算法本身的性能限制，以及硬件原因导致未能完全收敛。虽然在 ami33 和 ami49 电路上取得了较好的结果，但这是因为约束条件不同导致的，并不能作为参考。

比较 Pin 算法的有效性，表 II 展示了使用重心法

表 II
两个模型性能比较

Benchmarks	Model2		Model1	
	指标 1	指标 2	指标 1	指标 2
n10	53,481	46.56	32,339	51.10
n30	179,219	201.57	121,149	265.07
n50	263,335	781.42	184,986	748.25
n100	438,270	1965.04	350,740	1903.19
n200	878,356	8481.36	734,349	9366.06
ami33	67,261	36.79	48,669	115.94
ami49	236,010	231.95	210,168	155.14

(Model1) 和使用简单顺序引脚放置 (Model2) 的模型对比。可以看出, 重心法在 HPWL 的表现上明显优于简单顺序放置方法, 并且在运行时间上也没有明显增加。这表明了 Pin 引脚规划模块的有效性。

VI. 结论

本文针对 VLSI 布图中宏单元与 I/O 引脚的协同优化问题, 提出并实现了一种基于两阶段模拟退火的布图框架。该框架通过一个创新的“两阶段复合扰动”策略, 将基于序列对的宏单元布图探索与确定性的、基于重心的 I/O 引脚重规划紧密耦合, 在每次迭代中都对一个完整的布图方案进行评估和优化。

实验结果表明, 该方法能够在标准的 GRSC、MCNC 基准电路上获得高质量的布图解, 在总线长 (HPWL) 和芯片纵横比 (AR) 等关键指标上取得了有竞争力的结果, 验证了我们提出的协同优化思想的有效性。

除此之外, 我开源了自己的实验代码, 虽然目前代码项目处于初步阶段, 还有很多需要完善的地方, 但是由于其设计的松耦合以及各个基础部件相对完善, 其有很大的扩展潜力, 未来可以在此基础上进行更多算法的集成以及并行化的实现。

当然, 本工作也存在一些局限性。例如, 当前的成本函数未包含对 feedthrough 和时序的精确建模。未来的工作可以将更复杂的评估模型 (如我们讨论过的 FTAFP Feedthrough 估算) 集成到成本函数中, 并探索将模拟退火算法替换为更为先进的算法的可能性, 以进一步提升布图质量。

参考文献

- [1] E. B. N., A. A., B. M., G.-O. A., C. F., M. D., K. M., C. M., R. S., B. L., and K. L. S., “Hier-3d: A methodology for physical hierarchy exploration of 3-d ics,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 7, pp. 1957–1970, 2024, iEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems.
- [2] C. J., Z. W., and M. A. M., “A hybrid simulated annealing algorithm for nonslicing vlsi floorplanning,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 41, no. 4, pp. 544–553, 2011, iEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews) IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews).
- [3] J. Chen and W. Zhu, “A hybrid genetic algorithm for vlsi floorplanning,” in *2010 IEEE International Conference on Intelligent Computing and Intelligent Systems*, vol. 2, 2010, pp. 128–132.
- [4] S. B., V. R., A. B., K. K., I. V., and V. S., “A novel multicriteria optimization technique for vlsi floorplanning based on hybridized firefly and ant colony systems,” *IEEE Access*, vol. 11, pp. 14 677–14 692, 2023, iEEE Access IEEE Access.
- [5] M. A. and S. A., “Implementation of ai in the field of vlsi: A review,” in *2022 Second International Conference on Power, Control and Computing Technologies (ICPC2T)*, ser. 2022 Second International Conference on Power, Control and Computing Technologies (ICPC2T), 2022, pp. 1–5.
- [6] G. W., T. X., L. H., Z. Y., and Z. Y., “Thermal-aware fixed-outline 3-d ic floorplanning: An end-to-end learning-based approach,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 12, pp. 1882–1895, 2023, iEEE Transactions on Very Large Scale Integration (VLSI) Systems IEEE Transactions on Very Large Scale Integration (VLSI) Systems.
- [7] M. L. J., Y. H. C., and C. C. K., “Routability-driven floorplanning algorithm for mixed-size modules with fixed-outline constraint,” in *VLSI Design, Automation and Test(VLSI-DAT)*, ser. VLSI Design, Automation and Test(VLSI-DAT), 2015, pp. 1–4.
- [8] K. Kozminski, “Benchmarks for layout synthesis-evolution and current status.” New York, NY, USA: ACM, 1991, pp. 265–270, conference Proceedings.
- [9] M. H., F. K., N. S., and K. Y., “Rectangle-packing-based module placement,” in *Proceedings of IEEE International Conference on Computer Aided Design (ICCAD)*, ser. Proceedings of IEEE International Conference on Computer Aided Design (ICCAD), 1995, pp. 472–479.
- [10] N. B. R. B., R. T. A., S. P., D. M., and M. P., “Half-perimeter wirelength model for vlsi analytical placement,” in *2014 International Conference on Information Technology*, ser. 2014 International Conference on Information Technology, 2014, pp. 287–292.
- [11] F. HWANG, “An $O(n \log n)$ algorithm for rectilinear mhdml spanning trees,” 1979.
- [12] A. R. J. and L. M. I., “Seeing the forest and the trees: Steiner wirelength optimization in placement,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 4, pp. 632–644, 2007, iEEE Transactions on Computer-Aided

- [13] K. Deb, *Multi-objective Optimisation Using Evolutionary Algorithms: An Introduction*. London: Springer London, 2011, pp. 3–34. [Online]. Available: https://doi.org/10.1007/978-0-85729-652-8_1
- [14] W. Wei, L. Yanxiong, W. Zili, and C. Zhuyun, “Spotting chief speaker from press conference recordings based on silence detection,” in *2013 IEEE 11th International Conference on Electronic Measurement & Instruments*, vol. 1, 2013, pp. 147–150.
- [15] Q. Xu, S. Chen, and B. Li, “Combining the ant system algorithm and simulated annealing for 3d/2d fixed-outline floorplanning,” *Applied Soft Computing*, vol. 40, pp. 150–160, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494615006857>
- [16] H. Tashiro, A. Lau, J. Mori, N. Fujii, and Y. Kajikawa, “Email network analysis for leadership,” in *2011 IEEE International Conference on Industrial Engineering and Engineering Management*, 2011, pp. 1456–1460.
- [17] Z. Li, K. Tian, J. Zhai, Z. Li, S. Kai, S. Xu, B. Yu, and K. Zhao, *FTAFP: A Feedthrough-Aware Floorplanner for Hierarchical Design of Large-Scale SoCs*. New York, NY, USA: Association for Computing Machinery, 2025, p. 886–892. [Online]. Available: <https://doi.org/10.1145/3658617.3697728>